

PikaPlanner v2.1

현장 운용 · 전략 이해 · 스킬 확장 · 디버깅 가이드

Stable Candidate 기준 / Annotated Code 동봉

작성 기준: 2026-09-03 / 대회 공개 엔진 및 현재 v2.1 소스 기준

이 문서의 목적

이 문서는 “새 전략을 발명하는 문서”가 아니다. 현재 v2.1 이 왜 강한지, 어떤 가정 위에서 움직이는지, 어떤 종류의 실패가 버그이고 어떤 종류는 정상적인 경기 변동인지 이해한 뒤, 대회 현장에서 최소 수정으로 스킬을 연결하고 회귀 없이 제출하기 위한 운용 매뉴얼이다.

권장 사용법: 첫 10 분에는 1 장과 2 장만 읽고, 평소에는 3~16 장으로 코드를 숙지한다. 현장 스킬 공개 시에는 17~22 장을 체크리스트처럼 사용한다. 문제가 생기면 23 장의 증상별 진단표에서 시작한다.

목차

1. 1. 5 분 요약 - v2.1 을 한 문장으로 이해하기
2. 2. 현장용 절대 원칙과 제출 전 체크
3. 3. 공개 엔진에서 반드시 기억할 물리와 타이밍
4. 4. v2.1 의 전체 구조 - A~P 섹션 지도
5. 5. 좌우 canonicalization 과 x=216 비대칭
6. 6. 숨은 상태를 추정하는 방법
7. 7. 단일 공 물리 시뮬레이터와 collision 모델
8. 8. Reachability - 왜 full DP 대신 닫힌형식을 쓰는가
9. 9. 공격 평가함수 - landing 이 아니라 전체 trajectory 를 본다
10. 10. POWER 공격 planner
11. 11. Ground receive 와 v2.1 의 핵심 first-contact 수정
12. 12. Self-set 의 역할, 안전장치, touch-limit
13. 13. Jump / airborne / dive planner
14. 14. 상대 contact-first 방어와 reaction lock
15. 15. Tactical planner 의 우선순위
16. 16. Action safety executor - 전략보다 중요한 마지막 안전망
17. 17. 현재 알려진 약점과 의도적으로 남겨둔 것
18. 18. 두 테스트 상대를 어떻게 해석해야 하는가
19. 19. 회귀 테스트 설계와 승률 안정화 지표
20. 20. Replay 를 보고 실패 원인을 분류하는 법
21. 21. 스킬 공개 당일 확장 방법
22. 22. 스킬 유형별 패치 예시와 위험도
23. 23. 증상 -> 원인 -> 확인 위치 빠른 진단표
24. 24. 파라미터 튜닝 가이드
25. 25. 현장 시간대별 행동 계획
26. 26. 당부사항 - 하지 말아야 할 것
27. Appendix A. DEFAULT_CFG 전체 파라미터 사전
28. Appendix B. 주요 함수 사전
29. Appendix C. 테스트 명령어와 패키지 파일 구조

1.5 분 요약 - v2.1 을 한 문장으로 이해하기

PikaPlanner v2.1 은 “공의 낙하지점을 따라가는 봇”이 아니라, 앞으로 일어날 접촉(contact)을 계획하고 그 접촉 이후의 공 trajectory 를 직접 시뮬레이션해 행동을 선택하는 모델 기반 controller 다.

핵심 흐름은 아래 하나만 기억하면 된다.

snapshot

- > LEFT 기준으로 canonicalize
- > self/opp 의 숨은 vy 추정
- > 공의 정확한 free trajectory rollout
- > 현재 상황에 따라 contact / attack / defense 후보 생성
- > 전체 trajectory와 상대 reachability로 평가
- > planner intent 생성
- > stale held input을 막는 safety executor
- > 실제 LEFT/RIGHT action으로 반환

v2.1 의 가장 중요한 변화

v2 는 지상 리시브에서 “나중에 원하는 곳에서 공을 맞는 장면”을 평가했지만, 실제 엔진에서는 그보다 먼저 몸과 공이 겹치면 바로 충돌한다. v2.1 은 target 으로 이동하는 실제 held-action 경로를 따라가며 FIRST actual overlap 을 찾아 그 최초 접촉만 평가한다. 이 수정이 반복 local touch 와 5-touch 자멸을 크게 줄였다.

- 현재 강점: full-trajectory 공격 평가, cushion 이벤트 보존, opponent contact-first defense, reaction-lock 인식, airborne planner 분리, stale-input safety.
- 현재 핵심 가정: 빠른 JS 에서 실제 action latency 가 대체로 1 engine frame 이라는 것.
- 현재 남은 주의점: defense standby 후보가 상대 contact 전 “정말 도달 가능한 위치인지”는 hard constraint 가 아니고 move cost 로만 반영된다.
- 스킬 확장 주의점: SKILL.reachActions 는 현재 예약 함수지만 core 에서 호출되지 않는다. reach/hitbox 스킬이면 추가 wiring 이 필요하다.

2. 현장용 절대 원칙과 제출 전 체크

가장 중요한 원칙

Stable baseline 을 절대 덮어쓰지 않는다. 현장 수정은 항상 복사본에서 하고, 한 번에 하나의 가정만 바꾼다. “몇 판 이겼다”보다 touch-limit, self-score, 좌우 편향, blowout 빈도 같은 failure mode 가 사라졌는지를 먼저 본다.

2.1 제출 직전 반드시 확인

- `decide(snapshot)`가 top-level 에 그대로 존재한다.
- imports, fetch, DOM, 외부 파일 의존성이 없다.
- DEBUG=0, DEBUG_EXPORT=0 이다.
- 파일 크기가 4MB 보다 작다.
- verify_v2, cushion_regression, bench 를 마지막 수정본에 다시 실행했다.
- LEFT 와 RIGHT 를 모두 테스트했다.
- 최종 제출본을 별도 파일로 복사해 수정 금지 상태로 둔다.

2.2 “현재 잘 되는 것”을 보호해야 하는 이유

v2.1의 성능은 한 개의 똑똑한 평가함수에서 나오지 않는다. 물리 모델, contact timing, reachability, safety gate가 서로 맞물려 있다. 따라서 현장에서는 큰 아이디어를 추가하는 것보다 기존 invariant를 깨뜨리지 않는 것이 더 중요하다.

영역	안전한 수정	위험한 수정
SKILL.read / evaluate	새 필드 읽기, 게이지 가치 반영	필드 의미를 추측해서 core state를 바꿈
ballFrameHook	공개된 exact frame order가 있을 때만 반영	중력/속도를 임의 보정
extraActions	새 action을 후보로 추가	기존 planner 우선순위를 통째로 교체
DEFAULT_CFG	한 계열씩 작은 범위 sweep	여러 weight를 한 번에 수동 변경
executeSafely	새 action의 stale-input invariant 추가	기존 안전장치를 편의상 제거

3. 공개 엔진에서 반드시 기억할 물리와 타이밍

3.1 좌표와 히트박스

항목	값	해석
Court width	432	$x=0..432$
Net center	216	canonical 기준 분기점
Player ground y	244	player center
Ball ground y	252	ball center ground
Player half	32	AABB: $ dx \leq 32, dy \leq 32$
Walk	6 px/frame	입력 $x * 6$
Dive	8 px/frame	divingDirection 고정
Jump vy	-16	y 축 아래가 +
Gravity	+1/frame	공/플레이어
Bot snapshot cadence	3 frames	nominal 120ms
Engine FPS	25	40ms/frame

3.2 engine frame 과 bot decision 은 같은 것이 아니다

`tick`은 bot decision count 가 아니라 active game frame 마다 증가한다. snapshot 은 tick 3, 6, 9...에 요청되며, 빠른 Worker 는 보통 다음 engine frame 부터 새 action 이 적용된다. 따라서 “snapshot 3 프레임마다 받음”을 “항상 3 프레임 늦게 반응함”으로 이해하면 안 된다.

```

frame k:      old action -> physics/collision -> snapshot request (if k%3==0)
worker reply: call stack 이후 비동기 도착
frame k+1:    fast JS 라면 new action 적용 가능

=> v2.1 기본 ACTION_LATENCY = 1

```

Latency 스트레스 테스트의 의미

latency=2 시뮬레이션에서 문제가 생긴다고 해서 곧바로 ACTION_LATENCY=2 로 제출해야 한다는 뜻은 아니다. 실제 venue Chrome 의 JS Worker 응답 phase 가 우선이다. latency 를 바꾸면 ground first-contact, jump takeoff, power arming 의 시간축이 모두 달라진다.

3.3 player collision 보다 먼저 처리되는 것

공의 world update 가 먼저 일어나고 그 뒤 player collision 이 처리된다. 특히 어떤 frame 에서 ball 이 ground 판정을 받으면 그 frame 의 player overlap 으로 살릴 수 없다. v2.1 의 `firstGroundContactForTarget()`이 `bf.ground`에서 바로 null 을 반환하는 이유다.

3.4 power hit 의 정확한 수직 속도

```

ordinary collision 먼저:
  vy = -max(abs(preContactVy), 15)

power 적용:
  vy = abs(vy) * inputY * 2

=> |power vy| = 2 * max(abs(preContactVy), 15)

```

따라서 하방 smash($y=+1$)는 최소 +30 의 속도로 내려간다. 네트 근처에서 맞으면 2~4 프레임 안에 ground 에 도달할 수 있어 “친 뒤 보고 움직이는 수비”가 구조적으로 늦을 수 있다.

4. v2.1 의 전체 구조 - A~P 섹션 지도

섹션	역할	현장 수정 빈도
A	공개 엔진 상수	거의 금지
B	튜닝 파라미터	중간
C	SKILL adapter	가장 높음
D	memory/canonical utilities	낮음
E	player vertical state estimate	player skill 이면 높음
F	ball world physics	ball physics skill 이면 높음
G	collision response	collision skill 이면 높음
H	reachability	dash/hitbox skill 이면 높음
I	trajectory scoring	가능하면 유지
J	power attack planner	새 공격 skill 이면 일부
K	ground receive/self-set	리시브 문제 시
L	jump/airborne	player physics skill 이면
M	emergency dive	낮음
N	opponent threat/defense	수비 실패 반복 시
O	tactical dispatcher	가급적 유지
P	action safety	새 action 이면 반드시 검토

```

A constants -> C skill normalize
    |
D memory / E hidden state estimate
    |
F exact ball world + G collision
    |
H reachability -> I trajectory value
    |
J/K/L/M/N candidate planners
    |
O tactical priority
    |
P safety gate -> decide return

```

5. 좌우 canonicalization 과 x=216 비대칭

5.1 왜 내부를 항상 LEFT 로 만드는가

좌우에 대해 별도 전략 코드를 작성하면 테스트 양이 두 배가 되고, 한쪽만 고쳐지는 버그가 생긴다. v2.1 은 RIGHT 일 때 `x -> 432-x`, `vx -> -vx`, divingDirection 반전을 수행한다. 공격은 항상 +x, 상대는 항상 오른쪽이라고 생각하면 된다.

5.2 완전한 거울대칭이 아닌 한 점: net center

실제 엔진의 net-side 분기는 `ball.x < 216` 과 else 다. 따라서 정확히 x=216 이면 +|vx| 쪽으로 강제된다. RIGHT 를 LEFT 로 거울 반전하면 이 equality branch 가 그대로 대칭되지 않는다.

actual engine:

```
if (ball.x < 216) vx = -abs(vx)
else            vx = +abs(vx)
```

v2.1 canonical simulator:

```
x < 216 -> -abs
x > 216 -> +abs
x == 216 -> MEM.worldFlip 을 사용해 실제 원래 진영의 tie-break 복원
```

왜 중요했나

이 버그가 있으면 RIGHT 에서 실제로 자기 코트로 될 net-side shot 을 planner 가 상대 코트 winner 로 평가할 수 있다. 평균 성능이 좋아도 특정 geometry 에서 확정 자책이 생기므로 반드시 correctness bug 로 취급한다.

6. 숨은 상태를 추정하는 방법

6.1 snapshot 에 없는 값

- player yVelocity
- state2 의 delayBeforeNextFrame
- lyingDownDurationLeft
- collision edge flag
- 정확한 touch count
- Worker pending/실제 적용 frame

6.2 JUMP_TABLE / DIVE_TABLE 역추정

점프와 dive 의 수직 궤적은 결정론적이라 y 가 exact table 값과 맞으면 가능한 vy 후보를 찾을 수 있다. 상승/하강에서 같은 y 가 나올 수 있으므로 이전 snapshot 과의 trend 를 사용한다.

정확히 table 에 없는 경우는 snapshot 이 3 프레임 간격이라 상태전이가 중간에 끼었을 수 있다는 뜻이다. v2.1 은 $\frac{(y - \text{prevY})}{3}$ 의 finite difference 로 보수적으로 추정한다.

6.3 state2 hidden delay 는 왜 보수적으로 0 으로 두나

공격 candidate simulation 에서 state2 가 실제로 얼마나 더 유지되는지 snapshot 만으로 모른다. v2.1 은 shortest remaining delay 를 가정한다. 이 가정에서도 power contact 가 가능하면 실제 엔진의 armed window 가 더 길 경우 문제없다는 방향이다.

7. 단일 공 물리 시뮬레이터와 collision 모델

7.1 `stepBallWorld`는 single source of truth

공격 planner, self-set, defense threat, cushion scoring 이 서로 다른 물리를 사용하면 각각의 locally good decision 이 실제 경기에서는 충돌한다. v2.1 은 world physics 를 `stepBallWorld()` 하나로 통일한다.

30. futureX 를 보고 wall 반사
31. futureY < 0 이면 ceiling
32. 현재 x/y 로 net top/side 처리
33. SKILL.ballFrameHook

34. ground 검사
35. 위치 갱신
36. gravity 증가

7.2 이벤트를 지우지 않는 이유

trajectory는 매 frame 뿐 아니라 WALL_LEFT/RIGHT, NET_TOP, NET_SIDE, CEILING, GROUND 를 기록한다. cushion 은 별도 “특수샷 종류”가 아니라 동일한 물리 trajectory 안의 이벤트다. 따라서 새 벽샷을 위한 별도 if 문보다 기본 rollout 정확도가 중요하다.

7.3 ordinary collision 과 power collision

``ordinaryCollisionFrom()`` 이 두 경우의 공통 출발점이다. body hit 는 playerX 와 ballX 의 offset 으로 vx 를 만든다. exact center 는 실제 엔진 RNG 이므로 planner 에서는 3px 미만을 의도적으로 피한다.

8. Reachability - 왜 full DP 대신 닫힌형식을 쓰는가

8.1 문제 정의

공격 trajectory 의 어느 frame 에 상대가 AABB overlap 을 만들 수 있는지를 알아야 한다. 이를 모든 x/y/state/action 상태로 확장하면 조합 폭발이 일어난다.

8.2 v2.1 의 단순화가 가능한 이유

- 걷기에는 수평 관성이 없다. t 프레임 뒤 x reachable set 은 interval 로 표현 가능하다.
- 수직 이동은 ground, jump arc, dive arc 라는 작은 trajectory family 다.
- 이미 dive 중이면 direction 만 고정이고, recovery 후 다시 walk interval 이 열린다.
- 따라서 “t 에서 공과 겹칠 수 있나?”를 직접 계산하면 full game-tree 가 필요 없다.

8.3 상대 모델은 일부러 강하게 잡는다

상대 shot return 가능성을 평가할 때 새 jump 시작 frame 을 실제 bot scheduler phase 보다 넓게 허용한다. 이는 상대에게 유리한 over-approximation 이다. 공격이 “상대가 못 받는다”고 잘못 믿는 것보다 실제 강한 상대도 받을 수 있다고 보는 편이 robust 하다.

현재 미사용 config

``REACH_MAX_STATES``, ``REACH_X_QUANT`` 은 이전 full-state DP 설계의 흔적이며 현재 closed-form reachability 에서는 쓰이지 않는다. 현장에서 이 값을 튜닝해도 아무 변화가 없으므로 시간을 낭비하지 말 것.

9. 공격 평가함수 - landing 이 아니라 전체 trajectory 를 본다

9.1 핵심 평가 순서

37. 자기 코트에 떨어지면 거의 즉시 탈락시키는 큰 penalty
38. 상대가 전체 trajectory 어디에서도 못 건드리면 SHOT_UNRETURNABLE
39. 받을 수 있으면 earliest intercept 가 늦을수록 가점
40. 접촉 기회 수가 많을수록 감점
41. 상대 현재 위치와 landing 거리, edge, flight time 보조항

42. wall/net event 와 late reversal 소폭 보너스

43. score lead/trail 에 따른 미세 risk scaling

9.2 왜 landingX 만 보면 안 되는가

벽을 맞고 깊게 떨어지는 공도 중간에 상대 머리 위를 지나며 쉽게 맞을 수 있다. 반대로 landing 은 평범해 보여도 직전에 net-side reversal 이 생기면 반응 lock 때문에 어려울 수 있다. 따라서 earliest intercept 와 trajectory events 를 함께 봐야 한다.

9.3 weight 는 주역이 아니라 보조역

현재 성능의 핵심은 정확한 candidate trajectory 생성과 reachability 다. weight 를 조금 바꾸는 것보다 contact frame 을 1 프레임 틀리게 예측하는 것이 훨씬 큰 손실을 만든다. 현장에서는 정확도 버그를 weight tuning 으로 덮지 않는 것이 중요하다.

10. POWER 공격 planner

10.1 가능한 출력은 사실상 6 개

power hit 의 x 방향은 hitter side 가 결정하고, x 입력은 속도 크기 10/20 만 결정한다. $y = -1/0/+1$ 로 상향, 수평, 하향이 갈린다. 따라서 기본 공 출력은 slow/fast x up/flat/down 6 종이다. 다만 x 입력 부호는 armed window 동안 player 이동에는 영향을 준다.

10.2 `simulateImmediatePowerCandidate`의 중요성

현재 snapshot 에서 새 action 이 당장 같은 frame 에 적용되지 않는다. ACTION_LATENCY 동안 기존 held action 을 먼저 진행하고, 그 사이 이미 접촉했다면 새 candidate 를 무효로 한다. 이후 candidate 를 hold 하면서 실제 state2 overlap 이 생기는 순간을 찾는다.

이 방식은 “공 가까우면 hit 누르기”보다 훨씬 안전하다. hit 을 누른 frame 과 실제 collision frame 이 다를 수 있고, state2 는 여러 frame 유지되기 때문이다.

10.3 하방 smash gate

공이 네트에서 너무 멀면 down smash 가 자기 코트 ground 나 net-side rebound 로 이어질 수 있다.

`DOWN\_SMASH\_MAX\_NET\_DISTANCE` 는 후보 예산을 명백히 나쁜 하방샷에 쓰지 않게 하는 prior 다. 최종 판단은 여전히 exact trajectory 가 한다.

11. Ground receive 와 v2.1 의 핵심 first-contact 수정

11.1 v2 에서 무엇이 틀렸나

v2 는 미래 ball frame 과 이상적인 playerX 를 고른 뒤 “그 순간 이 위치에서 맞는다”고 평가했다. 하지만 그 target 으로 이동하는 동안 더 이른 frame 에서 이미 AABB overlap 이 생길 수 있었다. 실제 엔진은 첫 overlap 에서 속도를 바꾸므로 이후 계획은 전부 stale 해진다.

v2 내부 상상:

t=8 에 pPlayerX=130 에서 예쁜 CLEAR

실제:

t=5 에 이동 중 pPlayerX=112 에서 이미 overlap

-> local bump

-> 공이 자기 코트에 다시 툼

-> 다음 planner 도 또 CLEAR 라고 착각
-> 반복 touch -> 5-touch 실패 가능

11.2 v2.1 의 해결

44. 각 미래 low ball frame 에서 받을 targetX 후보 생성
45. targetX 를 계속 추적할 때 held action 기반 실제 x timeline 계산
46. 각 target 에 대해 FIRST AABB overlap 탐색
47. ground 가 먼저면 contact 불가 처리
48. 그 최초 contact 의 playerX 로 collision/trajectory 평가
49. hitbox edge 3px 계획은 scheduler jitter 에 취약하므로 제외

11.3 이 수정이 전략을 바꾼 것이 아닌 이유

CLEAR 와 SELF_SET 의 평가 철학, shot scoring, jump planner 는 그대로다. 바뀐 것은 “평가할 접촉이 실제로 일어나는 접촉인지”를 맞춘 것이다. 그래서 v2 의 좋은 branch 를 유지하면서 비정상적인 tail failure 를 제거하는 correctness patch 에 가깝다.

12. Self-set 의 역할, 안전장치, touch-limit

12.1 self-set 은 기본 전략이 아니다

v2.1 에서 self-set 은 deep incoming ball 을 첫 몸터치로 자기 코트 attack zone 근처에 띄울 수 있을 때만 제한적으로 고려한다. `SELF\_SET\_SCORE` 가 존재하지만 direct clear 보다 무조건 우선하는 구조가 아니다.

12.2 one-intent-per-possession gate

`selfSetPendingTick` 은 planner 가 SELF_SET 을 선택한 예상 contact ETA 를 기억한다. 그 시각이 지나고 공이 여전히 자기 코트면 `selfSetUsed=true` 로 만들어 같은 possession 에서 두 번째 intentional self-set 을 막는다.

12.3 touch estimate 는 참고치일 뿐

`ownTouchEstimate` / `oppTouchEstimate` 는 velocity change 와 근접 여부로 접촉을 추정하지만 중력 때문에 vy 는 정상 비행 중에도 변한다. 현재 safety-critical self-set gate 는 이 counter 에 의존하지 않는다.

현장에 꼭 기억할 것

`SELF\_SET\_MAX\_TOUCH\_EST` 는 현재 코드에서 사용되지 않는다. 숫자를 1 이나 0 으로 바꿔도 self-set 횟수 제한이 달라지지 않는다. 실제 gate 는 `selfSetUsed` 와 `selfSetPendingTick` 이다.

12.4 5-touch 를 다시 보면 무엇부터 볼까

- planner mode 가 실제로 SELF_SET 인지, 아니면 CLEAR 인데 local contact 가 반복됐는지
- first contact frame 과 실제 replay contact frame 이 일치하는지
- latency 가 1 보다 늦어 target timeline 이 어긋났는지
- 네트 crossing 이 snapshot 사이에 briefly 발생해 engine touch counter 가 reset 되었는지/아닌지
- 새 스킬이 contact 또는 possession 규칙을 바꾸었는지

13. Jump / airborne / dive planner

13.1 jump takeoff

지상에서 지금 점프했을 때 jump arc 와 future ball frame 이 겹치는 후보를 찾는다. 첫 3-frame held x 를 실제로 -1/0/+1 로 비교하고 남은 walk freedom 을 slack 으로 계산한다. ground receive 보다 `JUMP\_MIN\_VALUE\_GAIN` 이상 좋아야 점프한다.

13.2 airborne planner 를 분리한 이유

한 번 점프하면 수직 trajectory 는 committed 다. v2 초기에는 점프 후 ground receive planner 가 다시 target 을 바꿔 공에서 멀어지는 문제가 있었다. 현재는 state1/2 이면 `findAirborneIntercept()` 가 실제 현재 jump arc 에 맞춰 x 만 재계획한다.

13.3 dive

dive 는 walk 로는 늦고 dive 거리에는 들어오는 낮은 공에만 쓴다. recovery 와 lying state 비용이 매우 크므로 일반 movement shortcut 으로 쓰지 않는다.

주의

`chooseDive()` 는 여전히 비교적 간단한 거리 gate 다. vertical table 은 reachability 모델에는 들어가지만 emergency decision 자체는 full exact self simulation 보다 단순하다. 특정 dive miss 가 반복될 때는 먼저 replay 로 y-overlap 을 확인한다.

14. 상대 contact-first 방어와 reaction lock

14.1 공이 상대방에 있다고 current landing 을 믿으면 안 된다

상대가 곧 접촉할 수 있으면 현재 free trajectory 는 의미가 사라진다. 그래서 v2.1 은 상대 코트에서 가장 이른 reachable contact 를 먼저 찾고, 그 contact 에서 나올 수 있는 body/power trajectory 를 위협 집합으로 만든다.

14.2 ANTICIPATE

공이 상대방에 있고 opponent contact threat 가 존재하면 현재 공이 우리 쪽으로 돌아올 것처럼 보이더라도 jump/receive 를 commit 하지 않는다. 먼저 defensive standby 로 이동한다. 이는 pre-contact stale trajectory commit 을 막는 장치다.

14.3 reaction lock

상대가 친 순간부터 우리 새 informed action 이 적용될 때까지 2~4 프레임의 lock 이 생길 수 있다. 특히 contact 가 snapshot frame 과 같으면 snapshot 은 collision 전 상태이므로 첫 informed action 은 오히려 4 프레임 뒤가 된다.

```
contactFrame % 3 == 0 -> lock 4
contactFrame % 3 == 1 -> lock 3
contactFrame % 3 == 2 -> lock 2
```

14.4 maximin standby

각 candidate x 에서 모든 threat trajectory 를 얼마나 빨리 intercept 할 수 있는지 계산하고 최악의 threat score 가 가장 좋은 x 를 택한다. 특정 상대 shot 하나를 맞히는 prediction 보다 shot envelope 전체를 커버하려는 설계다.

14.5 현재 남은 약점

Known limitation

candidate standby x 가 “상대 contact 이전에 현재 위치에서 실제로 도달 가능한가”는 hard constraint 가 아니다. 현재는 $\text{abs}(x - \text{self}.x) * \text{DEFENSE_MOVE_COST}$ 로만 비용을 낸다. 빠른 네트 근처 하방샷을 1~4px 차이로 놓치는 장면이 반복된다면 가장 먼저 이 부분을 의심하되, 기존 maximin 을 통째로 바꾸지는 말 것.

15. Tactical planner 의 우선순위

1. POWER: 이미 공중이고 지금 power contact 가 실제로 가능하면 공격
2. COMMITTED: dive/lying/win/lose 는 neutral
3. ANTICIPATE: 공이 상대편 + 상대가 곧 contact 가능
4. OWN-SIDE BALL:
 - 4a. 이미 airborne -> AIR_INTERCEPT / AIR_RECOVER
 - 4b. ground receive 와 jump takeoff 비교
 - 4c. 둘 다 안 되면 DIVE
 - 4d. ground fallback
5. OPP-SIDE BALL: contact-first DEFEND

이 순서는 단순 if 문의 순서가 아니라 전략적 priority 다. 현장 스킬을 넣으면서 planCore 초반에 무조건 skill action 을 return 하면 POWER/COMMITTED safety 를 우회할 수 있다. 가능한 한 `extraActions` 와 `evaluate` 로 기존 후보 체계에 합류시키는 것이 좋다.

16. Action safety executor - 전략보다 중요한 마지막 안전망

16.1 왜 planner 와 executor 를 분리하나

action 이 다음 decision 까지 유지되기 때문에 현재 state 에서는 harmless 한 입력이 착지 후 다른 action 으로 해석될 수 있다. 예를 들어 공중에서 `hit=1, x=1` 을 유지하다 state0 이 되면 dive 가 발동할 수 있다.

16.2 현재 invariant

상태	허용/차단
state0	DIVE intent 가 아니면 hit=0; JUMP_INTERCEPT 가 아니면 y=-1 제거
state1/2	POWER 가 아니면 hit=0, y=-1 제거
착지 임박 POWER	hit 을 해제해 landing-frame accidental dive/rearm 방지
state2	이미 armed 이므로 hit=0; x/y 는 collision output 에 여전히 사용
state3/4/승패	neutral {0,0,0}

새 action 을 추가할 때

새 스킬 action 이 `hit`, `x`, `y` 조합을 재사용한다면 executeSafely 가 그 입력을 지워버리지 않는지 먼저 확인해야 한다. 반대로 safety gate 를 그냥 제거하면 stale-input 버그가 즉시 돌아올 가능성이 높다.

17. 현재 알려진 약점과 의도적으로 남겨둔 것

항목	상태	왜 아직 유지하는가
Defense standby reach hard constraint	남은 약점	넓게 수정한 실험에서 regression 위험이 있었음
Pre-jump defense	미구현	short smash에는 좋지만 deep return에 irreversible commit 위험
2-ply 일반 공격 recovery	제한적	serve 전용/구조 확장은 전략 변경 폭이 큼
Latency 2 robust receive	부분 취약	실측 fast JS latency1을 우선
Exact touch counter	없음	snapshot에 collision flag/touch count가 없고 heuristic은 신뢰 한계
SKILL.reachActions wiring	미연결	현재 skill 미공개; 실제 타입 확인 후 최소 patch가 안전
Unused config/helper	존재	기능 오류는 아니며 baseline 변경을 최소화

17.1 “버그”와 “모델 단순화”를 구분하기

x=216 tie-break, first-contact 누락처럼 실제 엔진과 planner가 다른 것은 버그다. 반면 opponent reachability를 상대방에게 유리하게 over-approximate하거나 defense standby를 빠른 analytic model로 계산하는 것은 의도된 근사다. 후자는 실제 로그에서 반복 실패가 확인되기 전까지 함부로 복잡하게 만들지 않는다.

18. 두 테스트 상대를 어떻게 해석해야 하는가

18.1 Opponent A - LuckyPunch v11

- 수비 standby에 도달한 뒤 Math.random 기반 좌우 진동을 넣어 위치/속도 외삽 공격을 일부러 깨뜨린다.
- 코드상 self-set은 MAX_SET_TOUCHES=0으로 꺼져 있다.
- 예측 수비와 serve 2-ply를 쓰는 공격형 상대라 deterministic stationary defender에만 과적합했는지 검사하기 좋다.
- 랜덤 진동 때문에 동일 strength라도 score streak가 흔들릴 수 있다.

18.2 Opponent B - LuckyPunch v16

- 경기 중 defense mode를 성적에 따라 선택한다.
- 리시브 contact timing을 MULTI/EARLY 사이에서 적응한다.
- 서브 공격을 immediate jump와 self-set 사이에서 결과에 따라 바꾼다.
- 뒤지고 있을 때 receive-set-second attack recovery state machine을 활성화한다.
- 따라서 한 세트 안에서도 상대 정책이 변하며, v2.1의 score 흐름이 갑자기 달라지는 것 자체는 버그 증거가 아니다.

18.3 v2.1 소규모 regression 신호

동일 latency=1, first-to-5 소규모 비교에서 Opponent B에 대해 base v2는 LEFT 1/3 + RIGHT 1/3, v2.1 candidate는 LEFT 3/3 + RIGHT 3/3 이었고 확인한 여섯 세트의 v2.1 touch-limit loss는 0이었다. 이 수치는 실제 대회 승률 예측이 아니라 “수정이 특정 실패를 줄였는지” 보는 회귀 지표로만 사용한다.

19. 회귀 테스트 설계와 승률 안정화 지표

19.1 평균 승률만 보면 안 된다

지표	의미	좋은 방향
Win rate	전체 strength	유지 또는 상승
Avg point diff	경기 지배력	상승
LEFT/RIGHT split	side-specific bug	격차 감소
Touch-limit losses	receive/contact correctness	0 에 가까움
Self-score power	physics/canonical 오류	0
Blowout loss rate	tail failure	감소
Seed 별 score variance	안정성	과도한 tail 감소
Latency2 stress	timing robustness	baseline 대비 악화 여부 확인

19.2 권장 매트릭스

Stage 1 - smoke

verify 900 calls
cushion regression
bench 2000 calls

Stage 2 - latency1

v2.1 vs Opp A, LEFT/RIGHT
v2.1 vs Opp B, LEFT/RIGHT
v2.1 vs base v2, LEFT/RIGHT

Stage 3 - latency2 stress

같은 매트릭스, 채택 기준은 승률보다 failure type

Stage 4 - 실제 Chrome

replay 가능한 seed/상황을 우선 수집

19.3 동일 seed paired comparison

새 버전과 baseline 을 서로 다른 random seed 로만 비교하면 variance 가 커진다. 가능한 한 같은 상대, 같은 방향, 같은 seed 묶음으로 결과를 짝지어 비교한다. 완전 동일 환경은 아니더라도 regression 신호가 훨씬 선명하다.

20. Replay 를 보고 실패 원인을 분류하는 법

20.1 한 실점마다 기록할 8 가지

50. 실점 직전 planner mode
51. self/opp state 와 y
52. ball x/y/vx/vy
53. 예상 contactFrame
54. 실제 첫 player-ball overlap frame
55. world event(net/wall/ground)
56. 현재 held action 과 새 action
57. touch count / point reason

20.2 대표 실패 패턴

관찰	가능성 높은 원인
planner CLEAR 인데 공이 자기 코트에 다시 뜸	planned contact 보다 earlier overlap / latency mismatch
RIGHT 에서만 net self-score	canonical net center/tie 또는 skill 좌우 변환
jump 시작 직후 반대편 공에 툴림	opponent contact gating 실패
jump 중 target 이 갑자기 ground landing 으로 바뀜	air/ground planner 경계 회귀
하방샷을 ground 직전 1~4px 로 놓침	standby feasibility/safety margin
착지 직후 갑자기 dive	stale hit+x safety gate
착지 직후 재점프	stale y=-1
5-touch 인데 SELF_SET mode 는 거의 없음	반복 accidental local contact 를 우선 의심

20.3 replay viewer

```
node tools/replay.mjs \
  --left ./PikaPlanner_v2_1_stable_candidate.js \
  --right ./opponents/LuckyPunch_v16_opponent.js \
  --seed 42 --score 10 --out ./tools/replay.json
```

그 다음 tools/replay_viewer.html 을 Chrome 에서 열고 replay.json 선택

DEBUG_EXPORT 가 켜진 simulator 에서는 mode, targetX, value, contactFrame 을 frame 별로 볼 수 있다. “결과가 나뻐다”보다 “planner 가 무엇을 믿고 있었나”를 확인하는 것이 핵심이다.

21. 스킬 공개 당일 확장 방법

21.1 먼저 스킬을 분류한다

스킬 유형	첫 수정 위치	추가 확인
새 정보/게이지	SKILL.read	좌우 canonical 필요 여부
가치/자원 관리	SKILL.evaluate	score scale 과 비교 가능 여부
새 tactical action	SKILL.extraActions	executeSafely 가 입력을 지우는지
emergency action	SKILL.emergencyOverride	기존 safety invariant 와 충돌
ball physics	SKILL.ballFrameHook	정확한 frame order 와 landing predictor
reach/hitbox	core reachability 추가 wiring	self + opponent 모두
player vertical physics	E/H/J/L/P 여러 곳	state estimate 와 exact self sim 동기화
collision/output	ordinaryCollisionFrom 주변	offense + opponent threat 모두 동기화

21.2 공개 직후 절대 하지 말아야 하는 것

- 필드 이름만 보고 의미를 추측
- 새 skill 을 무조건 사용하도록 planCore 맨 위에서 return
- 우리 skill 만 모델링하고 opponent skill threat 는 무시
- ball physics hook 만 고치고 expected contact/reachability 는 그대로
- 한두 판 결과만 보고 weight 를 연속 수정

21.3 새 snapshot field 의 canonicalization

예: skill 이 “오른쪽으로 dash 가능” 같은 world-direction 필드를 준다면 RIGHT 플레이어를 canonical LEFT 로 바꿀 때 방향 부호도 뒤집어야 한다. 게이지처럼 스칼라인 값은 그대로 읽으면 된다.

22. 스킬 유형별 패치 예시와 위험도

22.1 게이지 + 강한 일회성 공격

```
// SKILL.read
return { enabled: true, selfGauge: s.self.skillGauge, oppGauge: s.opp.skillGauge };

// SKILL.evaluate
if (candidate.type === "OUTGOING_TRAJECTORY" && ctx.skill.selfGauge >= COST) {
  // 실제 skill 사용 candidate 와 일반 shot 을 구분할 정보가 있을 때만 가점
}
return 0;
```

위험도는 낮지만, 일반 trajectory 에 gauge 가 있다는 이유만으로 가점을 주면 실제로 스킬을 사용하지 않은 샷까지 과대평가할 수 있다. candidate identity 를 분명히 해야 한다.

22.2 Dash / teleport

중요

`SKILL.reachActions()` 는 현재 호출되지 않는다. 이 함수만 채우면 아무 효과가 없다. opponent shot evaluation 의 `canPlayerReachBallAt()` 과 self receive/jump contact planner 양쪽에 실제 reach 변화가 들어가야 한다.

또한 실제 dash action 이 새로운 x/y/hit 조합을 사용한다면 `executeSafely` 의 clamp/gate 를 통과하는지 확인해야 한다.

22.3 공 중력/속도 변화

`ballFrameHook` 은 world update 중 net 처리 후, ground 검사 전에 호출된다. 공개 스킬의 실제 적용 순서가 이와 다르면 hook 위치 자체를 바꿔야 한다. 단순히 값을 맞추는 것보다 frame order 가 중요하다.

22.4 히트박스 확대

AABB half-size 가 변한다면 최소 세 군데가 같이 바뀌어야 한다: 실제 self overlap (`playerBallOverlapRaw` / firstGroundContact), opponent reachability (`xIntervalHits`, vertical overlap), emergency/dive geometry. 한 군데만 바꾸면 공격 평가와 실제 실행이 서로 다른 게임을 상상한다.

22.5 새로운 player state

state 7 같은 새 committed state 가 생기면 canonical snapshot 만 읽고 끝낼 수 없다. vertical estimate, neutral timeline, exact self simulator, tactical committed handling, safety executor 를 모두 점검한다.

23. 증상 -> 원인 -> 확인 위치 빠른 진단표

증상	1 순위 원인	확인 함수	우선 조치
5-touch 자멸	earlier accidental local contact	firstGroundContactForTarget / updateMemory	replay first overlap 비교
RIGHT 만 이상	canonical/tie/skill direction	canonicalize / stepBallWorld	x=216, 방향 필드 확인
네트 하방샷 반복 실패	standby feasibility/lock	chooseDefensiveStandby / groundStartInterceptScore	실제 contact 전 reachable x 비교
벽샷 예측 실패	physics event order	stepBallWorld	공개 코드와 frame order diff
power 가 발동 안 됨	state2 timing/latency	simulateImmediatePowerCandidate	old held action 기간 확인
power 후 자기 코트	collision/net model	ordinaryCollisionFrom / stepBallWorld	contact x,y, net event 확인
공중에서 공과 멀어짐	air planner regression	findAirborneIntercept	ground planner 가 끼는지 확인
무의미한 dive	emergency gate	chooseDive	y overlap + actual first contact 확인
CPU spike/timeouts	후보 폭/skill sim	predictOpponentThreats / extraActions	후보 수 제한, bench
새 skill 이 전혀 안 씀	혹 미연결/score<=0	extraActions/evaluate/reachActions	호출 경로 확인

24. 파라미터 튜닝 가이드

24.1 튜닝 순서

58. ACTION_LATENCY 와 실제 브라우저 timing 확인
59. receive/contact 계열
60. jump timing/gain
61. defense 위치/lock
62. power gating
63. shot evaluation 보조 weight

64. match-risk scale

24.2 한 번에 하나의 계열만

예를 들어 fast vertical shot 을 못 받는다고 `DEFENSE\_MOVE\_COST`, `HOME\_X`, `DIVE\_HORIZON`, `JUMP\_MIN\_VALUE\_GAIN` 을 동시에 바꾸면 어느 변화가 문제를 해결했는지 알 수 없다. 재현 seed 를 하나 확보하고 가장 직접적인 가정부터 수정한다.

24.3 weight 보다 hard correctness

`SHOT\_\*` weight 를 조절하기 전에 candidate trajectory 자체가 실제 엔진과 같은지 확인한다. v2.1 first-contact 사례처럼 trajectory 가 틀리면 weight 최적화는 틀린 시뮬레이터를 더 확신하게 만들 뿐이다.

25. 현장 시간대별 행동 계획

T-60 ~ T-30: baseline 고정

- stable_candidate SHA/복사본 확보
- verify/cushion/bench 결과 저장
- 두 상대 latency1 기본 결과 저장
- 실제 Chrome 에서 latency/리플레이 1~2 개 확보

스킬 공개 0~10 분

- 필드/규칙을 SKILL_DAY_PATCH_TEMPLATE.md 에 그대로 옮김
- 유형 분류
- 좌우/시간 순서/자원 소비 확인
- core 수정 필요 여부 판단

10~30 분: 최소 구현

- 한 종류의 hook 만 먼저 연결
- smoke test
- replay 에서 실제 skill activation 1 건 확인
- opponent skill 도 threat model 에 필요한지 확인

30 분 이후: 회귀와 채택

- Opp A/B 좌우 latency1
- base v2.1 direct regression
- touch-limit/self-score zero check
- latency2 stress
- 실제 브라우저 2~3 세트

마지막 10 분

- 새 아이디어 금지
- 버그 수정만 허용
- DEBUG off
- 파일명/용량/decide 확인

- 제출본 복사 후 더 이상 수정하지 않음

26. 당부사항 - 하지 말아야 할 것

65. “현재 점수 0:3 이니 전략이 망가졌다”라고 단정하지 말 것. 상대 B 는 경기 중 정책을 바꾸며, 상대 A 는 랜덤 진동을 쓴다.
66. 한두 개의 5:0 을 최고 전략의 증거로 보지 말 것. 짧은 first-to-10 게임은 variance 가 크다.
67. 정확한 contact bug 를 “공격성을 낮추자” 같은 전술 변화로 덮지 말 것.
68. 좌우 canonicalization 이 있으니 LEFT 만 테스트해도 된다고 생각하지 말 것. equality branch, skill direction, external state 에서 비대칭은 다시 생길 수 있다.
69. state2 에서 `hit=1` 이 계속 필요하다고 생각하지 말 것. 이미 armed 이면 collision output 에는 x/y 가 중요하고 hit 은 stale rearm 위험을 키울 수 있다.
70. 새 스킬이 강해 보여도 매 frame 무조건 쓰지 말 것. resource/cooldown 과 사용 후 recovery 가 있으면 전체 trajectory 가 치로 비교해야 한다.
71. 시뮬레이터의 승률을 실제 대회 승률로 읽지 말 것. simulator 는 브라우저 Worker timing, slow-motion/round transition 을 완전히 재현하지 않는다.
72. 마지막으로, 성능이 좋은 baseline 을 “더 우아한 코드”로 정리하려 하지 말 것. 현장에서는 작은 correctness patch 가 가장 높은 기대값을 가진다.

Appendix A. DEFAULT_CFG 전체 파라미터 사전

파라미터	현재값	역할	튜닝 위험	메모
ACTION_LATENCY	1	새 결정 적용까지 가정 frame	매우 높음	브라우저 실측 없이는 변경 금지
MOVE_DEADBAND	4	target 근처 steering 정지 폭	낮음	너무 작으면 진동
HOME_X	108	기본 수비 중심	중간	상대 분포 과적합 주의
BALL_HORIZON	96	공 rollout 최대 frame	낮음	너무 작으면 긴 cushion 누락
CONTACT_HORIZON	48	ground contact 탐색	중간	CPU/후반 contact tradeoff
THREAT_HORIZON	18	상대 contact 탐색	중간	짧으면 늦은 threat 누락
POWER_SEARCH_HOLD	8	power 후보 hold 탐색	중간	state2 window 와 관련
RECEIVE_OFFSETS	8,12,18,24,30	body hit player offset 후보	높음	outgoing vx geometry 직접 영향
SELF_SET_TARGET_X	170	self-set landing target	중간	attack zone 가정
SELF_SET_MAX_TOUCH_EST	2	현재 미사용	없음	튜닝해도 효과 없음
SELF_SET_DEEP_BALL_X	142	self-set 허용 incoming x	중간	너무 넓히면 local touch 증가
SELF_SET_SCORE	92	self-set 기본 가치	높음	직접 clear 와 비교됨
JUMP_MAX_CONTACT	27	jump contact 탐색 범위	중간	너무 길면 늦은 commit
JUMP_MIN_VALUE_GAIN	8	ground 보다 jump 가 좋아야 하는 차이	높음	낮추면 과점프
JUMP_POWER_MIN_CONTACT_FRAME	4	power 기대 최소 contact frame	중간	너무 이르면 arming 실패
DIVE_HORIZON	13	emergency dive lookahead	중간	과대하면 dive 남발
DIVE_MIN_BALL_Y	166	낮은 공에만 dive	중간	작게 하면 높은 공 dive
DIVE_EXTRA_MARGIN	5	walk/dive 경계 여유	중간	실전 latency jitter
SHOT_UNRETURNABLE	520	상대가 못 받는 shot 가치	높음	다른 score scale 기준
SHOT_RETURN_BASE	170	returnable shot 기준점	중간	전체 ranking scale
SHOT_INTERCEPT_FRAME_WEIGHT	7.5	늦은 intercept 가점	높음	공격 tempo
SHOT_INTERCEPT_OPTIONS_WEIGHT	1.6	많은 contact 기회 감점	중간	robust kill 선호
SHOT_LANDING_DISTANCE_WEIGHT	0.22	상대 위치에서 먼 landing 가점	낮음	보조항
SHOT_EDGE_WEIGHT	0.10	edge 가점	낮음	보조항
SHOT_TIME_WEIGHT	0.42	긴 flight 감점	낮음	tempo
SHOT_WALL_EVENT_BONUS	5	wall event 보너스	낮음	cushion 자체보다 reach 가 중요
SHOT_NET_TOP_EVENT_BONUS	8	net top 보너스	낮음	보조항
SHOT_NET_SIDE_EVENT_BONUS	10	net side 보너스	낮음	보조항
SHOT_LATE_REVERSAL_BONUS	5	intercept 직전 방향반전	낮음	reaction difficulty
SHOT_SELF_SIDE_PENALTY	20000	자기 코트 landing 금지	매우 높음	사실상 invariant
REACH_MAX_STATES	4500	현재 미사용	없음	legacy
REACH_X_QUANT	1	현재 미사용	없음	legacy
DEFENSE_X_STEP	8	standby 후보 간격	중간	작게 하면 계산량 증가
DEFENSE_MOVE_COST	0.30	현재 위치에서 먼 standby 비용	높음	현재 feasibility 약점과 관련
DEFENSE_NO_INTERCEPT_PENALTY	1000	막을 수 없는 threat penalty	높음	maximin hard 성격
DEFENSE_THREAT_LIMIT	24	위협 최대 개수	중간	CPU/coverage
DEFENSE_CONTACT_WINDOW	3	earliest 주변 contact 폭	중간	상대 timing uncertainty
POWER_NEAR_X	78	power planner 활성 거리 x	중간	너무 작으면 기회 누락
POWER_NEAR_Y	88	power planner 활성 거리 y	중간	후보 계산량/기회
DOWN_SMASH_MAX_NET_DISTANCE	100	하방 smash 사전 gate	높음	self-score 와 공격성
LANDING_SAFETY_FRAMES	3	착지 전 stale input 해제	높음	accidental dive/rejump
LEAD_SAFE	2	리드 risk scale 시작	낮음	경기 맥락
TRAIL_AGGRO	-2	뒤집 risk scale 시작	낮음	경기 맥락
LEAD_RISK_SCALE	0.92	리드 시 shot score scale	낮음	미세 조정
TRAIL_RISK_SCALE	1.08	뒤집 시 shot score scale	낮음	미세 조정

파라미터	현재값	역할	튜닝 위험	메모
DEBUG	0	콘솔 로그	없음	제출 0
DEBUG_EVERY	60	로그 간격	없음	개발용
DEBUG_EXPORT	0	replay debug export	없음	제출 0

Appendix B. 주요 함수 사전

함수	한 줄 역할
canonicalize	RIGHT 를 canonical LEFT 로 변환
updateMemory	possession/self-set/이전 상태 메모리
estimatePlayerVy	snapshot 에 없는 vy 추정
stepBallWorld	공 world physics 단일 진실원천
simulateTrajectory	frame/event rollout
ordinaryCollisionFrom	body/power collision 공통 응답
horizontalInterval	t 시점 수평 reachable interval
jumpVerticalOptionsAt	jump/ground 수직 overlap options
diveCanReachAt	dive reach 가능성
interceptTrajectory	전체 trajectory 상대 요격 분석
evaluateOutgoingTrajectory	공격 공통 score
simulateImmediatePowerCandidate	실제 power contact timing simulation
choosePowerAttack	공중 power 후보 선택
groundXTimelineToTarget	held-action receive 이동 timeline
firstGroundContactForTarget	FIRST actual ground contact
findBestGroundReceive	CLEAR/SELF_SET ground 후보 선택
findJumpTakeoff	지금 jump 할 가치 계산
findAirborneIntercept	점프 후 x 재계획
chooseDive	emergency dive
predictOpponentThreats	상대 future contact 에서 threat 생성
reactionLockAfterOpponentContact	post-shot 반응 lock
groundStartInterceptScore	standby x 의 threat 방어 score
chooseDefensiveStandby	maximin 수비 위치
planCore	전술 dispatcher
executeSafely	stale input 안전 gate
decide	대회 entry point

Appendix B.1 현재 “있지만 사실상 사용하지 않는” 항목

- `SELF\_SET\_MAX\_TOUCH\_EST`: config 에 있지만 현재 gate 에서 미사용
- `REACH\_MAX\_STATES`, `REACH\_X\_QUANT`: 이전 DP 설계 흔적, 현재 미사용
- `selfCanReachX()`: 현재 core 에서 호출되지 않는 helper
- `SKILL.reachActions()`: 예약 훅이지만 현재 core 에서 호출되지 않음
- `ownTouchEstimate/oppTouchEstimate`: 갱신되지만 현재 v2.1 핵심 safety 판단의 기준으로 사용하지 않음

Appendix C. 테스트 명령어와 패키지 파일 구조

C.1 기본 명령

```
node tools/verify_v2.mjs ./PikaPlanner_v2_1_stable_candidate.js
node tools/cushion_regression.mjs ./PikaPlanner_v2_1_stable_candidate.js
node tools/bench.mjs ./PikaPlanner_v2_1_stable_candidate.js 2000
```

C.2 대전

```
node tools/run_series.mjs \
  --left ./PikaPlanner_v2_1_stable_candidate.js \
  --right ./opponents/LuckyPunch_v16_opponent.js \
  --matches 20 --seed 31001 --latency 1
```

반드시 파일을 바꿔 RIGHT 역할도 별도로 돌린다. `run\_field\_matrix.mjs`는 Opponent A/B 와 base v2 를 양쪽 방향으로 묶어 확인하기 위한 편의 스크립트다.

C.3 파일 구조

```
PikaPlanner_v2_1_Field_Package/
PikaPlanner_v2_1_stable_candidate.js # 제출 기준선
PikaPlanner_v2_1_annotated.js        # 읽기/현장 수정용 주석판
PikaPlanner_v2_1_Field_Guide.pdf     # 이 문서
FIELD_DAY_CHECKLIST.md
SKILL_DAY_PATCH_TEMPLATE.md
opponents/
  LuckyPunch_v11_opponent.js
  LuckyPunch_v16_opponent.js
reference/
  PikaPlanner_v2_reference.js
  V2_1_DIAGNOSTIC.md
tools/
  sim_engine.mjs / run_series.mjs / replay.mjs / ...
```

마지막 요약

제출 전 기억할 세 문장

1) v2.1 의 강점은 평가 weight 보다 “실제 일어날 contact 와 trajectory 를 맞추는 것”에서 나온다. 2) 현장 스킬은 core 를 갈아엎지 말고 SKILL adapter 에서 시작하되, reach/player/collision 을 바꾸는 스킬은 관련 simulator 전체를 동기화해야 한다. 3) 승률 한 숫자보다 touch-limit, self-score, 좌우 비대칭 같은 tail failure 가 0 에 가까운지를 먼저 확인한다.

이 문서와 annotated code 는 baseline 을 이해하고 빠르게 수정하기 위한 도구다. 실제 공개 스킬의 필드/규칙이 나오면 그 사실이 최우선이며, 이 문서의 예시는 공개되지 않은 기능을 추측한 것이 아니다.